

计算机组成原理大作业报告

实验名称：多周期 CPU 与五级流水线

学号：2410668

姓名：吕昊远

班次：张金、董前琨

一、实验目的

1. 理解多周期 CPU 与五级流水线 CPU 的结构、工作流程以及二者的差异。
2. 学习了解流水线中数据相关的成因，理解填充空指令、流水线阻塞、数据旁路三种不同的解决方案的原理与差异。
3. 重点掌握数据旁路（forwarding）的硬件实现方法，并能在 Verilog 中正确实现前递通路与相应的相关检测逻辑。
4. 能够编写 MIPS 汇编计算程序，主动构造并观察数据相关，通过仿真验证功能的正确性与优化效果。
5. 进一步加强运用 Verilog 进行电路设计、调试与仿真验证的能力。

二、实验内容

1. 将提供的多周期 CPU 与五级流水线 CPU 代码配置为工程，修复其中的 Bug，复现并验证其汇编程序的执行结果。
2. 编写一段会产生数据相关的完整 MIPS 计算程序，放入指令存储器，并在五级流水线 CPU 上运行。
3. 选择数据旁路（forwarding）方案解决数据冲突，修改流水线 CPU 核心模块，加入前递通路与控制逻辑；通过仿真对比”阻塞”与”旁路”两种方案的数据相关停顿拍数与对应的执行结果。
4. 对于流水线的优化，分析旁路对流水线吞吐与性能的提升，并进行正确性回归验证。

三、实验原理

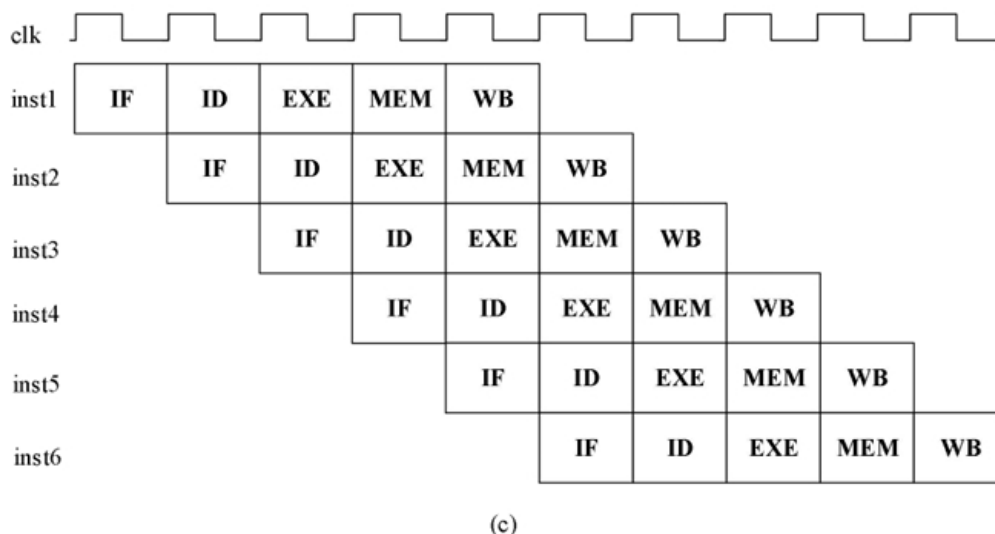


图 1: 实验指导书-图 9.1(c)-五级流水 CPU 时空图

1. 五级流水线 CPU 结构

五级流水线将一条指令的执行拆分为取指 (IF)、译码/取操作数 (ID)、执行 (EXE)、访存 (MEM)、写回 (WB) 五个阶段，相邻指令在不同阶段上重叠执行，从而在理想情况下达到接近每周期完成一条指令的吞吐。本实验所用流水线 CPU 的源操作数在 ID 级从寄存器堆读出，运算在 EXE 级完成，访存在 MEM 级完成，结果在 WB 级写回寄存器堆。与多周期 CPU 不同，流水线 CPU 带有延迟槽，分支/跳转目标以延迟槽指令的 PC（即分支指令 PC+4）为基准进行计算。

2. 数据相关

当一条指令需要用到前面指令尚未写回寄存器堆的结果时，就产生了数据相关。由于本实验中流水线的寄存器堆是同步写、异步读的设计，一条指令的结果需要它走到 WB 级，在时钟上升沿写入寄存器堆之后，后续指令才能在 ID 级读到正确值。而如果后续指令与之相隔太近，在它读寄存器时生产指令还停留在 EXE 或 MEM 级，就会导致读到旧值，进而引起计算错误。这种“写后读” (RAW) 是流水线中最为常见的数据相关。

3. 三种解决方案对比

在实验要求中，主要提供了三种解决数据相关的思路，不同的解决方案实现起来有所不同，在效果上也具有很大差异。

- **填充空指令 (NOP):** 是一种软件方案，由编译器/程序员在相关指令之间手工插入若干条 `nop`，把生产指令与消费指令的距离拉开到结果已写回为止。这种方法无需改动硬件，但会大量增大代码体积、浪费指令槽，且高度依赖人工或编译器来保证逻辑的正确。

- **流水线阻塞 (stall/wait)**: 硬件方案。译码级检测到数据相关时, 令当前指令暂不进入下一级 (插入气泡) 并冻结在其之前的各级, 直到相关解除。这种方法无需改动程序, 但是每次相关都要付出停顿拍数, 会对性能造成一定的损失。
- **数据旁路 (forwarding/前递)**: 硬件方案。把后续流水级中已经算出但尚未写回寄存器堆的结果, 通过专门的数据通路直接送回译码级作为源操作数, 使绝大多数 RAW 无需停顿即可获得正确数据。这种方法在性能上是三者中最好的, 但需要额外的数据通路 with 选择控制逻辑来实现。

本实验选择**数据旁路**方案, 其原理与指导书中的原理图所指的流水线优化一致 (图 9.1c)。

4. 旁路 (forwarding) 的实现原理

指令在 ID 级需要使用某寄存器时, 依照由新到旧的优先级依次检查 EXE 级、MEM 级、WB 级是否正在写该寄存器, 命中则取该级的结果作为操作数。优先级为 **EXE > MEM > WB**。同样地, 因为寄存器堆同步写、异步读, WB 级在本周期写入的值, ID 级在同一周期内读不到, 因此也需要从 WB 级 (即 `rf_wdata`) 前递。

而对于一些必要的停顿, 也需要保留相应的阻塞。有两类结果在对应级还未完成全部流程, 不能从该级前递, 必须使用阻塞来实现:

- `load` 指令的数据要到 MEM 级才从数据 RAM 返回, 本实验同步 RAM 下还要再等一个周期, 故 `load` 在 EXE 级不可前递;
- `mflo/mfhi/mfc0` 的值要到 WB 级才确定, 因而在 EXE/MEM 级也不能前递。

此外, 在这种改进方案下, 分支也同样可以受益。流水线的分支判断、`jr` 目标、ALU 操作数、`store` 数据都直接使用 ID 级的源操作数, 因此只要把源操作数也前递化, 分支与运算就都能用上前递结果, 从而无需为它们单独停顿。

5. 演示程序

为了清晰地观察数据相关并对比两种方案, 本实验选取具有直线依赖的程序进行实现。类似于之前单周期 CPU 实验中的程序, 这里我们也选择计算斐波那契数列这一任务, 从 `fib(1)...``fib(10)` 逐项计算数列的值。在计算过程中, 每一条 `addu` 都直接用到紧邻上一条指令的结果, 因此在阻塞方案下每一步都会触发停顿, 而在旁路方案这些停顿则会被完全消除。最后将计算出的 `fib(10)=55 (0x37)` 写入数据 RAM 的 `0x80` 字节地址, 以便于核对结果。

四、实验过程

4.1 多周期 CPU 与五级流水线的调试与复现

将源码的 `.mif` 需要转化为 `.coe`, 依照指导书步骤进行代码的导入以及 IP 核配置, 最终的仿真波形会出现问题, 即老师在实验要求中提及的 Bug。经过分析可知, 该问题是由 IP 核的配置导致的。其正确配置方式如下图:

Component Nameinst_rom

Basic

Port A Options

Other Options

Summary

Interface TypeNative

Generate address interface with 32 bits

Memory TypeSingle Port ROM

Common Clock

ECC Options

ECC TypeNo ECC

Error Injection PinsSingle Bit Error Injection

Write Enable

Byte Write Enable

Byte Size (bits)9

Algorithm Options

Defines the algorithm used to concatenate the block RAM primitives. Refer datasheet for more information.

AlgorithmMinimum Area

Primitive8kx2

Component Nameinst_rom

Basic

Port A Options

Other Options

Summary

Memory Size

Port A Width32Range: 1 to 4608 (bits)

Port A Depth256Range: 2 to 1048576

The Width and Depth values are used for Read Operation in Port A

Operating ModeWrite First

Enable Port TypeAlways Enabled

Port A Optional Output Registers

Primitives Output Register

Core Output Register

SoftECC Input Register

REGCEA Pin

Port A Output Reset Options

RSTA Pin (set/reset pin)

Output Reset Value (Hex)0

Reset Memory Latch

Reset PriorityCE (Latch or Register Enable)

READ Address Change A

Read Address Change A

图 2: IP 核: inst_rom

Component Namedata_ram

Basic

Port A Options

Port B Options

Other Options

Summary

Interface TypeNative

Generate address interface with 32 bits

Memory TypeTrue Dual Port RAM

Common Clock

ECC Options

ECC TypeNo ECC

Error Injection PinsSingle Bit Error Injection

Write Enable

Byte Write Enable

Byte Size (bits)8

Algorithm Options

Defines the algorithm used to concatenate the block RAM primitives. Refer datasheet for more information.

AlgorithmMinimum Area

Primitive8kx2

Component Namedata_ram

Basic

Port A Options

Port B Options

Other Options

Summary

Memory Size

Write Width32Range: 8 to 4096 (bits)

Read Width32

Write Depth256Range: 2 to 1048576

Read Depth256

Operating ModeWrite First

Enable Port TypeAlways Enabled

Port A Optional Output Registers

Primitives Output Register

Core Output Register

SoftECC Input Register

REGCEA Pin

Port A Output Reset Options

RSTA Pin (set/reset pin)

Output Reset Value (Hex)0

Reset Memory Latch

Reset PriorityCE (Latch or Register Enable)

READ Address Change A

Read Address Change A

图 3: IP 核: data_ram

5	0x40	addu	\$4,\$2,\$3	00432021	; F4	[RAW: \$3 来自上一条]
6	0x44	addu	\$5,\$3,\$4	00642821	; F5	[RAW: \$4]
7	0x48	addu	\$6,\$4,\$5	00853021	; F6	[RAW: \$5]
8	0x4C	addu	\$7,\$5,\$6	00A63821	; F7	[RAW: \$6]
9	0x50	addu	\$8,\$6,\$7	00C74021	; F8	[RAW: \$7]
10	0x54	addu	\$9,\$7,\$8	00E84821	; F9	[RAW: \$8]
11	0x58	addu	\$10,\$8,\$9	01095021	; F10 = 55	[RAW: \$9]
12	0x5C	sw	\$10,0x80(\$0)	AC0A0080	; Mem[0x80] = F10 = 55	
13	0x60	beq	\$0,\$0,self	1000FFFF	; 自旋停机	
14	0x64	nop		00000000	; 延迟槽	

将上述汇编程序改写为.coe 文件，导入到 inst_rom 的 IP 核配置中。

ii. 仿真文件

为了逐时钟周期观察流水线各级 PC 与数据相关的情况，也需要对 testbench 进行一定的修改。在每个时钟沿打印各级 PC 与相关检测信号 rs_wait/rt_wait，并在寄存器写回/内存写时打印事件，统计 ID 级因为数据相关而停顿的总周期数。

```

1 always @(posedge clk) if (resetn) begin
2     cyc = cyc + 1;
3     if (uut.ID_module.rs_wait | uut.ID_module.rt_wait)
4         stall_cnt = stall_cnt + 1; // 统计数据相关停顿拍数
5     if (uut.rf_wen && uut.rf_wdest!=0) // 寄存器写回事件
6         $display("cyc%0d WB=%h RFWR $%0d=%h", cyc, WB_pc[11:0], uut.rf_wdest, uut.
7             rf_wdata);
8     if (uut.dm_wen!=0) // 内存写事件
9         $display("cyc%0d MEM=%h MEMW [%h]=%h", cyc, MEM_pc[11:0], uut.dm_addr
10             [11:0], uut.dm_wdata);
11 end

```

iii. 阻塞仿真结果

在原始的阻塞流水线上运行程序，得到如下结果：

```

1 cyc7 WB=034 RFWR $1=00000001
2 cyc9 WB=038 RFWR $2=00000001
3 cyc13 WB=03c RFWR $3=00000002
4 cyc17 WB=040 RFWR $4=00000003
5 cyc21 WB=044 RFWR $5=00000005
6 cyc25 WB=048 RFWR $6=00000008
7 cyc29 WB=04c RFWR $7=0000000d
8 cyc33 WB=050 RFWR $8=00000015
9 cyc37 WB=054 RFWR $9=00000022
10 cyc41 WB=058 RFWR $10=00000037
11 cyc44 MEM=05c MEMW [080]=00000037

```

观察输出结果，数列各项 2,3,5,8,13,21,34,55 均正确，结果 Mem[0x80]=0x37(55)，与预期一致，原始的阻塞方案功能正确；但在波形图中也能看到，运行过程中每条相关指令都会触发停顿，整个程序运行下来共计有 26 个时钟周期的停顿，这意味着数据相关为性能带来了较大的影响，这也是我们接下来需要优化的问题。

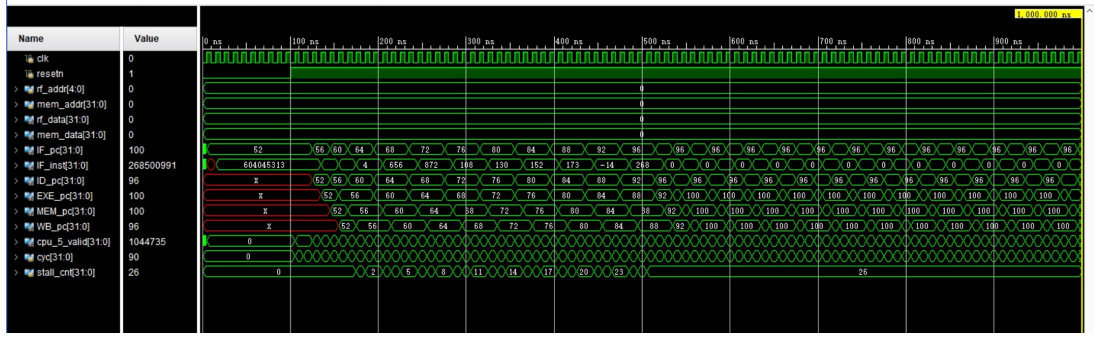


图 6: 阻塞仿真波形图

4.3 旁路方案的实现

分析程序运行流程可知，被等待的结果其实在 EXE 级已由 ALU 算出，只是尚未写回至寄存器堆；与其阻塞等待，不如直接把它从所在级前递回译码级，这就是旁路方案的核心解决思路。在具体实现上主要需要修改 `exe.v`、`mem.v`、`decode.v`、`pipeline_cpu.v` 这几个文件。以下是详细的实现过程。

i. `exe.v`

在模块端口列表末尾新增两个输出端口，用来表示 EXE 级前递的信息。

```
1 output [31:0] EXE_result, // EXE级前递值
2 output EXE_fwd_ok, // EXE级结果是否可前递
```

同时新增对应的赋值（`exe_result` 即 EXE 级已算出的写回结果；`load` 数据未到、`mfhi/mflo/mfc0` 值在 WB 才出，因此不可前递）：

```
1 assign EXE_result = exe_result;
2 assign EXE_fwd_ok = EXE_valid & ~(mem_control[3] | mfhi | mflo | mfc0);
3 // mem_control[3]=load
```

ii. `mem.v`: 引出 MEM 级前递信息

与 `exe` 类似，在端口列表末尾新增输出端口，这里用来表示 MEM 级前递的信息；以及对应的赋值语句（`mem_result` 为 `load` 结果或透传的 EXE 结果；`mfhi/mflo/mfc0` 值在 WB 才出，`load` 需等 `MEM_over`）：

```
1 output [31:0] MEM_result, // MEM级前递值
2 output MEM_fwd_ok, // MEM级结果是否可前递

1 assign MEM_result = mem_result;
2 assign MEM_fwd_ok = MEM_valid & ~(mfhi | mflo | mfc0) & (~inst_load | MEM_over);
```

iii. `decode.v`

这部分主要实现前递选择与相关检测的改写。首先修改原来的源操作数输入端口为 `rs_value_raw/rt_value_raw` 表示寄存器堆原始读出值，在模块内则用 `rs_value/rt_value` 来做前递选择，方便其他地方的改动。然后在端口列表中新增 5 个前递输入，分别与之前的改动相对应。

```

1  input    [31:0] rs_value_raw,  // 第一源操作数(寄存器堆原始读出)
2  input    [31:0] rt_value_raw,  // 第二源操作数(寄存器堆原始读出)

```

```

1  input    [31:0] EXE_result,    // EXE级前递值
2  input    [31:0] MEM_result,    // MEM级前递值
3  input    [31:0] WB_result,     // WB级前递值(=rf_wdata)
4  input                EXE_fwd_ok, // EXE级结果可前递
5  input                MEM_fwd_ok, // MEM级结果可前递

```

在模块内继续新增前递选择的条件判断, 使用同名的 `rs_value/rt_value` 来输出前递后的值。

```

1  // 前递优先级 EXE > MEM > WB; 0号寄存器恒为0不前递; 都未命中则用原始读值
2  wire [31:0] rs_value, rt_value;
3  assign rs_value = (rs!=5'd0 & rs==EXE_wdest & EXE_fwd_ok) ? EXE_result :
4                      (rs!=5'd0 & rs==MEM_wdest & MEM_fwd_ok) ? MEM_result :
5                      (rs!=5'd0 & rs==WB_wdest )                ? WB_result :
6                      rs_value_raw;
7  assign rt_value = (rt!=5'd0 & rt==EXE_wdest & EXE_fwd_ok) ? EXE_result :
8                      (rt!=5'd0 & rt==MEM_wdest & MEM_fwd_ok) ? MEM_result :
9                      (rt!=5'd0 & rt==WB_wdest )                ? WB_result :
10                     rt_value_raw;

```

最后再将原来的相关检测逻辑进行修改, 改为只有最近的生产层级的结果不可前递时才停顿(命中 WB 一定可前递, 这里不用计入停顿):

```

1  wire rs_hit_exe = (rs==EXE_wdest);
2  wire rs_hit_mem = (rs==MEM_wdest) & ~rs_hit_exe;  // EXE未命中才看MEM
3  wire rt_hit_exe = (rt==EXE_wdest);
4  wire rt_hit_mem = (rt==MEM_wdest) & ~rt_hit_exe;
5  assign rs_wait = ~inst_no_rs & (rs!=5'd0)
6                  & ( (rs_hit_exe & ~EXE_fwd_ok) | (rs_hit_mem & ~MEM_fwd_ok) );
7  assign rt_wait = ~inst_no_rt & (rt!=5'd0)
8                  & ( (rt_hit_exe & ~EXE_fwd_ok) | (rt_hit_mem & ~MEM_fwd_ok) );

```

iv. pipeline_cpu.v

新增前递线网声明:

```

1  wire [31:0] EXE_result, MEM_result;
2  wire                EXE_fwd_ok, MEM_fwd_ok;

```

`decode` 例化部分将源操作数端口与我们之前的修改对齐, 并新增前递输入连接 (WB 级前递值直接连接已有的写回数据 `rf_wdata`); `exe` 和 `mem` 的例化也需新增前递输出的连接,

```

1  .rs_value_raw (rs_value ), // 原 .rs_value
2  .rt_value_raw (rt_value ), // 原 .rt_value
3  .EXE_result   (EXE_result),
4  .MEM_result   (MEM_result),
5  .WB_result    (rf_wdata ), // WB前递值 = 写回数据
6  .EXE_fwd_ok   (EXE_fwd_ok),
7  .MEM_fwd_ok   (MEM_fwd_ok),

```

```
1      .MEM_result      (MEM_result),
2      .MEM_fwd_ok      (MEM_fwd_ok),
```

到这里整个前递通路修改完成, exe/mem 送出结果与对应的标志, pipeline_cpu 连接到 decode, 再由 decode 选出前递后的操作数, 并仅在取不到值时 (load/mflop) 才发生停顿。

4.4 旁路方案仿真结果

使用完全相同的演示程序在旁路改造后的流水线上运行, 得到如下输出与波形图:

1	cyc7	WB=034	RFWR	\$1=00000001
2	cyc9	WB=038	RFWR	\$2=00000001
3	cyc11	WB=03c	RFWR	\$3=00000002
4	cyc13	WB=040	RFWR	\$4=00000003
5	cyc15	WB=044	RFWR	\$5=00000005
6	cyc17	WB=048	RFWR	\$6=00000008
7	cyc19	WB=04c	RFWR	\$7=0000000d
8	cyc21	WB=050	RFWR	\$8=00000015
9	cyc23	WB=054	RFWR	\$9=00000022
10	cyc25	WB=058	RFWR	\$10=00000037
11	cyc26	MEM=05c	MEMW	[080]=00000037

得到的各项数列的值与最终结果均与阻塞方案一致 (Mem[0x80]=0x37=55), 计算结果正确。但不同的是, 数据相关停顿从 26 个时钟周期直接降为 0, 各项结果均连续逐周期写回, 程序运行到 `sw` 的用时从约 44 个时钟周期缩短到约 26 个时钟周期。

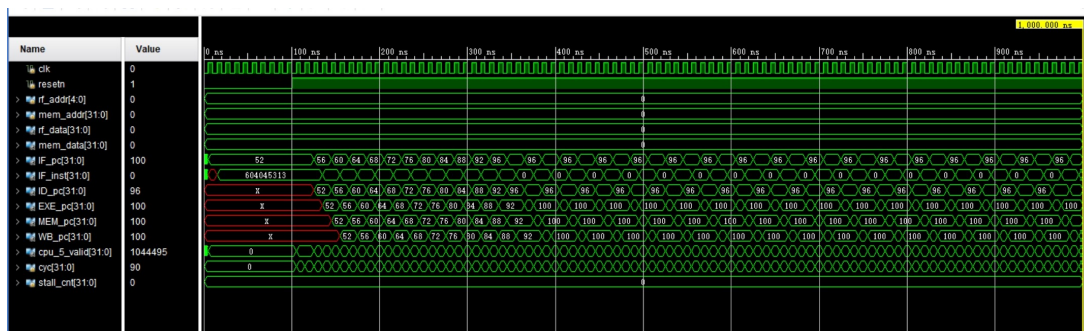


图 7: 旁路仿真波形图

4.5 指令验证

为确认旁路改动没有破坏其它指令的正确性，将源码中原来的那段程序（包含 load、分支、乘法、异常处理、字节 store 等共 45 条指令）在改造后的流水线上运行进行测试，与原始的阻塞流水线逐条比对寄存器与内存写事件。

经验证, 两者写事件完全一致, 旁路方案对全部 45 条指令的执行结果与原始流水线逐条均相同。此外, 在相同的 4000 个时钟周期的仿真内, 阻塞方案共计完成了 1263 条写事件, 而旁路方案则完成了 1511 条, 大幅提高了吞吐量。

上述结果表明，旁路并不会改变任何指令执行的结果，在保证正确性的同时，可以消除大部分停顿，在相同时间内完成更多的指令，从而达到了性能优化的目的。

五、总结感想

本次大作业在复现多周期与五级流水线 CPU 的基础上，重点完成了流水线数据相关的分析与旁路优化的实现。实验中选取了斐波那契数列这一数据依赖性较强的计算程序，其中密集的写后读相关也方便在仿真中清楚地观察停顿；随后通过修改相应的设计文件以及检测逻辑，实现了数据旁路，将同一程序的数据相关停顿从 26 个时钟周期降为 0，同时还对源码自带的全指令集程序逐条进行比对，验证了这一优化的正确性。

通过这次实验，我对流水线 CPU 的级间数据流动、数据相关的本质，以及三种处理数据相关的方案有了深入理解；尤其是认识到旁路实现中容易被忽视的两个细节：寄存器堆同步写异步读导致的 WB 级也需前递，以及 load/mflo 这类结果较晚的指令必须保留停顿。在调试与仿真验证过程中，逐周期追踪流水线各级状态、比对确认正确性的方法，也提高了我的 Verilog 代码编写能力与对指令的熟悉程度，对于硬件设计的测试与验证积累了宝贵的经验。